FRONTEND ARCHITECTURE BLUEPRINT

Enterprise UI Structure & Framework Design

Hospital Management System (HMS) Single Page Application

Author: Senior UI Architect

UI Technology Stack: React 18+, Tailwind CSS v3+, React Router v6

Document Version: 2.0.0 (Incremented Blueprint)

Date: June 12, 2026

Classification: Internal Tech Spec

1. Global UI Shell Layout Architecture

The Hospital Management System UI is structured as a responsive, high-performance web platform utilizing a single-instantiated workspace pattern. The application viewport is divided into three primary regions: a fixed lateral navigation panel (Sidebar), a contextual top banner (Navbar), and a fluid scrollable canvas (Main Content).

1.1 Global Navbar Component Items

The Navbar resides persistently across all authenticated spaces, serving as the system's global utility hub. It contains:

- **Left Region:** Contextual Breadcrumbs mapping the nested active route (e.g., `Patients > Records > Electronic Health Record`) and a toggle trigger to fold/collapse the Sidebar navigation.
- **Center Region:** Global Omnibox Search bar optimized for hotkey access (`Ctrl + K`) allowing rapid context switching to pull up Patient records, Medical IDs, or Active Rooms.
- **Right Region (System Controls):**
  - **Emergency Hotbutton:** High-visibility flash trigger executing immediate global triage code requests.
  - **Real-time Notification Bell:** Houses an interactive dropdown wired to WebSockets for priority updates (Lab anomalies, incoming admissions, critical alerts).
  - **User Profile Avatar Component:** Displays user picture, active role badge (e.g., `DOCTOR`), and opens a panel containing `Profile Settings`, `Switch Tenant/Department`, and `Logout` actions.

2. Role-Based Sidebar Menu Structure

Sidebar items are dynamically rendered and filtered by the client layout layer upon token decoding. Navigation elements are grouped cleanly into semantic sections.

User Role	Rendered Sidebar Menu Structure (Hierarchical)
Super Admin	• Dashboard (Global Infrastructure Health) • Tenant Management (Hospitals, Clinics, Branches) • Global Configuration (System Settings, Database Archival Logs) • Subscription & License Audit
Admin	• Dashboard (Operational & Financial KPIs) • Staff Directory (Hiring, Onboarding, Shift Rosters) • Department Hubs (ICU, Emergency, Outpatient Allocation) • Audit Trails (Security Logs, Resource Utilization)
Doctor	• Dashboard (My Rounds, Patient Count, Critical Flags) • Appointment Planner (Calendar View, Consultation Slots) • Patient EMR Registry (Vitals, Clinical History, SOAP Notes) • E-Prescriptions & Clinical Orders (Lab Requests, Imaging)
Nurse	• Dashboard (Ward Map, Shift Handover Summary) • Inpatient Census (Bed Allocation, Triage Tracking) • Medication Administration (MAR Checklists, Infusion Schedules) • Vital Sign Input Logs & Incident Reports
Receptionist	• Dashboard (Check-in Queue, Daily Walk-ins) • Dynamic Scheduling Matrix (Doctor Availability, Booking Panel) • Patient Registration File (Demographics, Insurance Scans) • Outpatient Queue Queue Management
Lab Technician	• Dashboard (Pending Orders Counter, High-Priority Stat Flags) • Sample Collection Inventory (Barcode Scans, Tracking) • Analysis Workflow Matrix (Hematology, Radiology Uploads) • Verification Portal (Approve & Dispatch Results)
Pharmacist	• Dashboard (Prescription Queue, Dispensing Desk) • Inventory Core (Stock Levels, Cold-Chain Warnings, Expiry Matrix) • Order Fullfilment Pipeline (Verification, Billing Hold Checks) • Vendor Logistics (Procurement, Formularies)
Billing Executive	• Dashboard (Accounts Receivable, Daily Claims Summary) • Invoice Workspace (Discharge Bills, Itemized Outpatient Receipts) • Claims Processing (Insurance Verification, Pre-Auth Approvals) • Payment Gateway Audits
Patient	• Dashboard (My Health Summary, Next Appointment Notice) • Medical Records (Download Prescriptions, Lab Reports) • Appointment Booking Portal (Telehealth vs Physical visits) • Billing History & Online Payments Statement

3. Standard Role Dashboard Specifications

Dashboards follow an adaptive grid framework matching user personas, displaying critical items inside highly visible Tailwind components.

> **Design System Rule:** All dashboard grids must prioritize information scanning. Core metrics should reside on top, flanked by graphical trend analytics, with a deep transactional grid at the base.

3.1 Doctor Clinical Command Dashboard Layout

- **Row 1: Analytic Metric Cards (Grid 4-Cols)**
  - Card 1: Active Inpatients (Count, trend indicator)
  - Card 2: Pending Consultations Today
  - Card 3: Critical Lab Results Awaiting Review (Color coded soft-red flag)
  - Card 4: Telehealth Sessions Slotted
- **Row 2: Dual Workspaces (Grid 2-Cols / Split 60-40)**
  - Left Area (60%): Live Timeline Schedule Grid showing chronologically active patient appointments with quick-launch action triggers for EHR files.
  - Right Area (40%): Task Priority Checklist & Urgent Inpatient Ward Flags (e.g., Bed 402 Alert).

3.2 Patient Portal Dashboard Layout

- **Row 1: Welcome Header Card**
  - Displays check-in status, QR code for automated kiosk check-in, and notice banner for upcoming appointment (e.g., `Cardiology appointment at 10:30 AM with Dr. Jenkins`).
- **Row 2: Functional Insights Grid (3-Cols)**
  - Card 1: Recent Prescriptions Widget (Direct link to view/print code parameters).
  - Card 2: Latest Diagnostic Reports (Flagging complete collections).
  - Card 3: Outstanding Statements (Payment link wrapper with quick checkout integrations).

4. Application Route Structure & Navigation Security

The routing layer uses `react-router-dom` browser navigation bindings configured declaratively. Route endpoints are guarded by a Higher-Order Component configuration pattern analyzing claims in the state wrapper.

4.1 Abstract Declarative Route Tree

```
/ (Public Root)
├── /login                              # Global Authentication Portal
├── /forgot-password                    # Credential Recovery Workflow
└── /dashboard (Authenticated Shell Layout)
    ├── /admin (Admin Route Wrapper - Roles: ['ADMIN', 'SUPER_ADMIN'])
    │   ├── /staff                      # Directory List
    │   └── /wards                      # Infrastructure Allocation
    ├── /doctor (Doctor Route Wrapper - Roles: ['DOCTOR'])
    │   ├── /appointments               # Slot Scheduling Layout
    │   └── /patients                   # Patient List Lookup
    │       └── /:patientId             # Dedicated Patient File (Nested EHR Tabs)
    ├── /nurse (Nurse Route Wrapper - Roles: ['NURSE'])
    │   ├── /beds                       # Ward Tracking
    │   └── /medication                 # Daily MAR Records Pipeline
    └── /patient (Patient Route Wrapper - Roles: ['PATIENT'])
        ├── /records                    # Diagnostic History Logs
        └── /book                       # Schedule Wizard Grid
```

5. Standard React Folder Architecture (Scalable Modular Pattern)

The directory layout follows a Domain-Driven modular packaging standard combined with a strong shared UI base to simplify development and isolate features cleanly.

```
src/
├── assets/                          # Static Assets
│   ├── images/                      # Logos, Icons, Static Diagram Illus.
│   └── styles/                      # Global Configurations
│       └── tailwind.css             # Base directives with custom extended palettes
│
├── components/                      # Global Shared UI Components (Atomic Design)
│   ├── ui/                          # Pure Presentation Components (Design System primitives)
│   │   ├── Button.tsx               # Tailwind-styled button component variations
│   │   ├── Input.tsx                # Extracted accessible input element fields
│   │   ├── Modal.tsx                # Flexible dialog system component wrappers
│   │   ├── Table.tsx                # Paginated dataset presentation table view grid
│   │   └── Badge.tsx                # Color-coded metric labels (Status, priority flags)
│   ├── feedback/                    # Interaction elements (Toasts, Spinners)
│   └── navigation/                  # Layout framing elements
│       ├── Navbar.tsx               # Top contextual menu strip
│       └── Sidebar.tsx              # Dynamic menu shell layout
│
├── context/                         # React Context Providers for State Scope
│   ├── AuthContext.tsx              # Security contexts tracking token life and role schemas
│   ├── ThemeContext.tsx             # Context managing accessibility/dark layouts
│   └── SocketContext.tsx            # Persistent real-time server connections
│
├── features/                        # Domain-Specific Scopes (Co-located Code)
│   ├── patient-emr/                 # Encapsulated EMR Feature Module
│   │   ├── components/              # Internal components unique to EMR
│   │   │   ├── SoapNotesForm.tsx    # Internal clinical notes editor view
│   │   │   └── VitalsCard.tsx       # Real-time dashboard indicator graphs
│   │   ├── hooks/                   # Custom data lifecycle logic pipelines
│   │   │   └── usePatientEmr.ts     # Coordinates data flow for single patients
│   │   ├── services/                # Backend HTTP API connection wrappers
│   │   │   └── emrApi.ts            # Specific CRUD paths mapping endpoints
│   │   └── index.ts                 # Public API file exposing selected sub-features
│   │
│   ├── scheduling/                  # Scheduling & Booking Feature Module
│   └── billing/                     # Invoice Processing & Claims Module
│
├── hooks/                           # Global React Functional Pipeline Custom Extensions
│   ├── useAuth.ts                   # Abstract wrapper hook reading user profiles
│   ├── useDebounce.ts               # Logic delay regulator used for search omniboxes
│   └── useNotification.ts           # Trigger hook for layout status elements
│
├── layouts/                         # Shared Layout Frame Structural Foundations
│   ├── AuthLayout.tsx               # Layout frame wrapper for login panels
│   └── DashboardLayout.tsx          # Standard authenticated shell layout grid
│
├── routes/                          # Navigation Gateways and Route Orchestration
│   ├── AppRoutes.tsx                # Central route registry layout element tree
│   ├── ProtectedRoute.tsx           # Security Guard Component checking privileges
│   └── rolesConfig.ts               # Declarative list mapping paths to arrays of roles
│
├── services/                        # Base Platform Integration Clients
│   ├── apiClient.ts                 # Core Axios instantiation with auto interceptor hooks
│   └── storage.ts                   # Safe abstract logic reading sessionStorage
│
├── utils/                           # Pure Deterministic Utility Helpers
│   ├── validators.ts                # Structural evaluation tools
│   └── formatters.ts                # Date parsing, text normalizers, currency converters
│
```

```
├── App.tsx                          # Application Main Setup Entry Point Canvas
└── main.tsx                         # Root DOM Bootstrapper Mounting Script
```